

MÁQUINA AUTÔNOMA DE VENDAS – Arquitetura Completa

Expansão do fluxo da Fábrica Agêntica de Publicações para uma máquina de vendas 24/7.

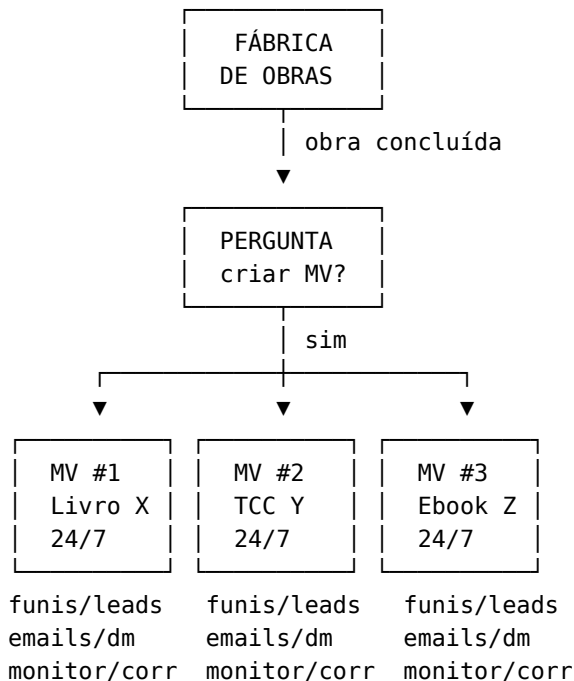
Base teórica: Russel Brunson (Hook-Story-Offer, Value Ladder, Funnel Scripts).

1. CONCEITO CENTRAL: MÁQUINAS INDEPENDENTES POR OBRA

Após a finalização de cada obra (livro, TCC, e-book, etc.), o sistema exibe uma **pergunta**:

Obra "{título}" concluída com sucesso.
Deseja criar uma MÁQUINA DE VENDAS autônoma para esta obra?
[1] Sim – criar máquina completa
[2] Sim – criar máquina parcial (escolher etapas)
[3] Não – apenas salvar a obra

Cada máquina é independente: sua própria escada de valor, seus funis, seus leads, suas campanhas, seus subagentes. O operador pode ter N máquinas rodando em paralelo, cada uma com seu ciclo de vida autônomo.



Metadados de cada máquina

Cada máquina criada gera um manifesto em `marketing/maquinas/{slug}/manifesto.json`:

```
{
  "id": "mv-20260808-observabilidade",
  "slug": "observabilidade-sistemas-distribuidos",
  "obra_origem": "output/livros/observabilidade-sistemas-distribuidos/",
  "tipo_obra": "livro",
  "criada_em": "2026-08-08T10:00:00Z",
  "status": "ativa",
  "harness_detectado": "mimocode",
  "modelos_disponiveis": ["mimo-v2.5-pro", "mimo-v2.5-lite", "claude-sonnet"],
  "escada_valor": {...},
  "funis": [...],
  "subagentes_ativos": [...],
  "metricas": {...}
}
```

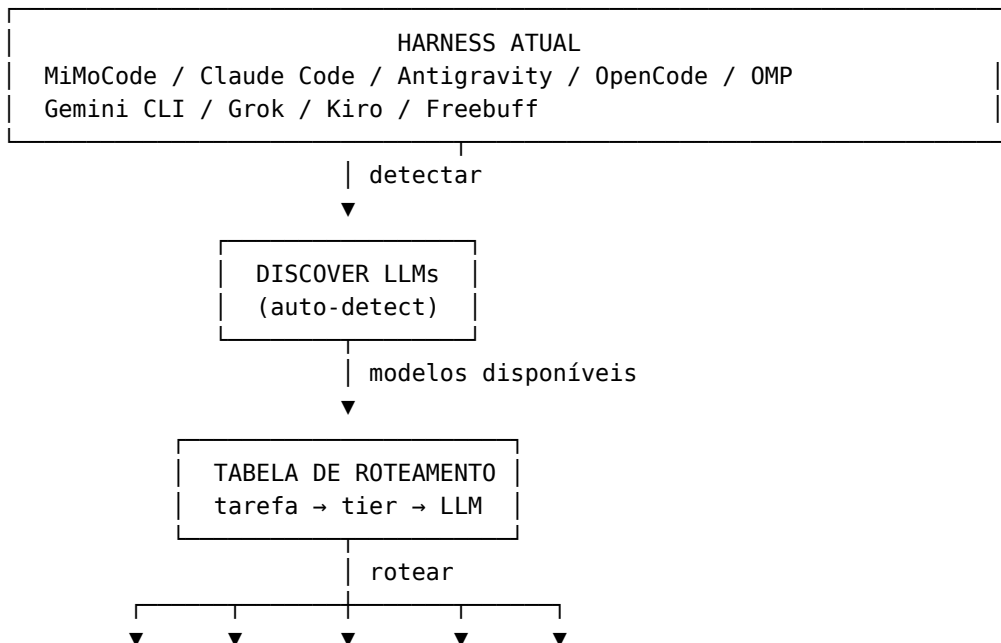
2. DESCOBERTA DINÂMICA DE LLMs (ROTEAMENTO POR HARNESS)

O Problema

Cada harness (MiMoCode, Claude Code, Antigravity, Freebuff, OpenCode, OMP, Gemini CLI, Grok, Kiro) possui LLMs diferentes configuradas. Hardcodar modelos é desperdício — um subagente de scoring de leads não precisa de MiMo V2.5 Pro quando Haiku ou Lite faz o mesmo por 10x menos.

A Solução: Detecção Dinâmica + Roteamento por Tarefa

O sistema detecta automaticamente quais modelos estão disponíveis no harness e roteia cada subagente para o **modelo mais barato capaz de executar a tarefa**.



[tier1]	[tier2]	[tier3]	[tier4]	[API]
lite	standard	pro	heavy	ext

Script: scripts/descobrir_modelos.py

```
#!/usr/bin/env python3
"""Detecta harness atual e lista LLMs disponíveis."""
import json, os, subprocess, sys

HARNESS_SIGNATURES = {
    "mimocode": [".mimocode", "mimocode.json"],
    "claude_code": [".claude", "CLAUDE.md"],
    "antigravity": [".antigravity", "antigravity.json"],
    "opencode": [".opencode", "opencode.json"],
    "freebuff": [".freebuff", "freebuff.json"],
    "omp": [".omp", "omp.json", ".ohmypi"],
    "gemini_cli": [".gemini", "gemini.json", ".gemini-cli"],
    "grok": [".grok", "grok.json", ".xai"],
    "kiro": [".kiro", "kiro.json", ".kiro/"],
}

TIER_MAP = {
    # Tier 1: Lite – tarefas simples (scoring, classificação, templates)
    "lite": [
        "claude-haiku", "claude-3-5-haiku",
        "mimo-v2.5-lite", "mimo-lite",
        "gpt-4o-mini", "gpt-3.5-turbo",
        "gemini-flash", "gemini-2.0-flash", "gemini-2.5-flash",
        "grok-1", "grok-2-mini", # xAI Grok (leve)
        "amazon-titan-lite", # Kiro/Bedrock
        "mistral-small", "llama-3-8b", # OMP (multi-provider)
    ],
    # Tier 2: Standard – copy, análise, redação
    "standard": [
        "claude-sonnet", "claude-sonnet-4", "claude-3-5-sonnet",
        "mimo-v2.5", "mimo-v2.5-standard",
        "gpt-4o", "gpt-4-turbo",
        "gemini-pro", "gemini-2.5-pro",
        "grok-2", "grok-3", # xAI Grok (padrão)
        "claude-sonnet-v2", # Kiro/Bedrock
        "mistral-large", "llama-3-70b", # OMP (multi-provider)
    ],
    # Tier 3: Pro – estratégia, raciocínio complexo
    "pro": [
        "claude-opus", "claude-opus-4",
        "mimo-v2.5-pro",
        "gpt-4", "o1", "o3",
        "gemini-ultra", "gemini-2.5-pro-deep-think",
        "grok-3-heavy", # xAI Grok (avançado)
        "claude-opus-v2", # Kiro/Bedrock
    ],
}
```

```

        "deepseek-r1", "qwen-max",                                # OMP (multi-provider)
    ],
    # Tier 4: API externa – TTS, imagem, vídeo
    "external_api": [
        "elevenlabs", "openai-tts", "google-tts",
        "dall-e-3", "midjourney", "ideogram",
        "imagen-3",                                     # Google (Gemini)
        "aurora",                                       # xAI (Grok)
        "amazon-titan-image",                           # Kiro/Bedrock
        "heygen", "synthesia", "remotion",
    ],
}

def detectar_harness():
    cwd = os.getcwd()
    for harness, signatures in HARNESS_SIGNATURES.items():
        for sig in signatures:
            if os.path.exists(os.path.join(cwd, sig)) or \
               os.path.exists(os.path.expanduser(f"~/{sig}")):
                return harness
    return "desconhecido"

def listar_modelos_disponiveis(harness):
    """Consulta config do harness para listar modelos."""
    if harness == "mimocode":
        for path in ["~/.mimocode/config.json", ".mimocode.json"]:
            p = os.path.expanduser(path)
            if os.path.exists(p):
                with open(p) as f:
                    cfg = json.load(f)
                return cfg.get("models", cfg.get("providers", {}))
    elif harness == "claude_code":
        if os.environ.get("ANTHROPIC_API_KEY"):
            return {"anthropic": TIER_MAP["lite"] + TIER_MAP["standard"] +
                    TIER_MAP["pro"]}
    elif harness == "antigravity":
        for path in ["~/.antigravity/config.json", ".antigravity.json"]:
            p = os.path.expanduser(path)
            if os.path.exists(p):
                with open(p) as f:
                    cfg = json.load(f)
                return cfg.get("models", {})
    elif harness == "opencode":
        for path in ["~/.opencode/config.json", ".opencode.json"]:
            p = os.path.expanduser(path)
            if os.path.exists(p):
                with open(p) as f:
                    cfg = json.load(f)
                return cfg.get("models", cfg.get("providers", {}))

```

```

elif harness == "omp":
    # OMP (Oh My Pi): multi-provider – lê de omp.json ou ~/.omp/config.json
    for path in [".omp.json", "~/.omp/config.json", "~/.ohmypi/config.json"]:
        p = os.path.expanduser(path)
        if os.path.exists(p):
            with open(p) as f:
                cfg = json.load(f)
                return cfg.get("models", cfg.get("providers", {}))
    # Fallback: detectar por env vars
    modelos = {}
    if os.environ.get("OPENAI_API_KEY"):
        modelos["openai"] = ["gpt-4o-mini", "gpt-4o", "o3"]
    if os.environ.get("ANTHROPIC_API_KEY"):
        modelos["anthropic"] = ["claude-haiku", "claude-sonnet", "claude-
opus"]
    if os.environ.get("GOOGLE_API_KEY"):
        modelos["google"] = ["gemini-flash", "gemini-pro", "gemini-ultra"]
    return modelos
elif harness == "gemini_cli":
    # Gemini CLI: usa GOOGLE_API_KEY ou GOOGLE_GENAI_API_KEY
    if os.environ.get("GOOGLE_API_KEY") or
os.environ.get("GOOGLE_GENAI_API_KEY"):
        return {"google": ["gemini-2.0-flash", "gemini-2.5-flash",
"gemini-2.5-pro", "gemini-ultra"]}
    for path in [ "~/.gemini/settings.json", ".gemini/settings.json"]:
        p = os.path.expanduser(path)
        if os.path.exists(p):
            with open(p) as f:
                cfg = json.load(f)
                return cfg.get("models", {"google": cfg.get("availableModels",
[])}))
elif harness == "grok":
    # Grok (xAI): usa XAI_API_KEY
    if os.environ.get("XAI_API_KEY"):
        return {"xai": ["grok-2-mini", "grok-2", "grok-3", "grok-3-heavy"]}
    for path in [ "~/.grok/config.json", ".grok.json"]:
        p = os.path.expanduser(path)
        if os.path.exists(p):
            with open(p) as f:
                cfg = json.load(f)
                return cfg.get("models", {})
elif harness == "kiro":
    # Kiro (Amazon): usa AWS Bedrock – models via ~/.kiro/config ou env
    for path in [ "~/.kiro/config.json", ".kiro/settings.json"]:
        p = os.path.expanduser(path)
        if os.path.exists(p):
            with open(p) as f:
                cfg = json.load(f)
                return cfg.get("models", cfg.get("bedrock_models", {}))

```

```

        # Fallback Bedrock: Claude + Titan
        if os.environ.get("AWS_ACCESS_KEY_ID"):
            return {"bedrock": [
                "amazon-titan-lite", "claude-haiku",
                "claude-sonnet-v2", "claude-opus-v2"
            ]}
        return {}

def rotear_tarefa(tarefa, modelos_disponiveis):
    """Retorna o modelo mais barato capaz para a tarefa."""
    tier_necessario = TAREFA_TIER.get(tarefa, "lite")
    candidatos = TIER_MAP.get(tier_necessario, TIER_MAP["lite"])

    for modelo in candidatos:
        for provider, modelos in modelos_disponiveis.items():
            if modelo in modelos or any(modelo in m for m in modelos):
                return {"provider": provider, "model": modelo}

    # Fallback: primeiro modelo disponível
    for provider, modelos in modelos_disponiveis.items():
        if modelos:
            return {"provider": provider, "model": modelos[0]}

    return {"provider": "default", "model": "inherit"}

# Mapeamento: tarefa → tier mínimo necessário
TAREFA_TIER = {
    "qualificar_leads": "lite",
    "scoring_leads": "lite",
    "gerar_template_email": "lite",
    "classificar_conteudo": "lite",
    "gerar_copy": "standard",
    "escrever_emails": "standard",
    "gerar_pagina_venda": "standard",
    "analise_funil": "standard",
    "gerar_artes_prompt": "standard",
    "gerar_roteiro_video": "standard",
    "estrategia_marketing": "pro",
    "definir_escada_valor": "pro",
    "diagnosticar_gargalo": "pro",
    "otimizar_campanha": "pro",
    "gerar_audio": "external_api",
    "gerar_imagem": "external_api",
    "gerar_video": "external_api",
}

if __name__ == "__main__":
    harness = detectar_harness()
    modelos = listar_modelos_disponiveis(harness)

```

```

resultado = {
    "harness": harness,
    "modelos_disponiveis": modelos,
    "roteamento": {}
}
for tarefa in TAREFA_TIER:
    resultado["roteamento"][tarefa] = rotear_tarefa(tarefa, modelos)
print(json.dumps(resultado, indent=2, ensure_ascii=False))

```

Tabela de Roteamento (config/roteamento_modelos.json)

```

{
  "_descricao": "Roteamento de LLMs por tier de tarefa. O sistema detecta o harness e usa o modelo mais barato disponível para cada tier.",
  "tiers": {
    "lite": {
      "descricao": "Tarefas simples: scoring, classificação, templates básicos",
      "custo_relativo": "muito baixo",
      "modelos_preferidos": [
        "claude-haiku", "mimo-v2.5-lite", "gpt-4o-mini", "gemini-flash",
        "grok-2-mini", "amazon-titan-lite", "mistral-small"
      ],
      "tarefas": [
        "qualificar_leads",
        "scoring_leads",
        "gerar_template_email",
        "classificar_conteudo",
        "deduplicar_leads",
        "formatar_output"
      ]
    },
    "standard": {
      "descricao": "Tarefas intermediárias: copy, análise, redação de conteúdo",
      "custo_relativo": "médio",
      "modelos_preferidos": [
        "claude-sonnet", "mimo-v2.5", "gpt-4o", "gemini-pro",
        "grok-2", "claude-sonnet-v2", "mistral-large"
      ],
      "tarefas": [
        "gerar_copy",
        "escrever_emails",
        "gerar_pagina_venda",
        "gerar_pagina_captura",
        "analise_funil",
        "gerar_artes_prompt",
        "gerar_roteiro_video",
        "gerar_sequencia_dm",
        "escrever_post_social"
      ]
    }
  }
}

```

```

    "pro": {
      "descricao": "Tarefas complexas: estratégia, diagnóstico, raciocínio profundo",
      "custo_relativo": "alto",
      "modelos_preferidos": [
        "claude-opus", "mimo-v2.5-pro", "o3",
        "grok-3-heavy", "claude-opus-v2", "deepseek-r1"
      ],
      "tarefas": [
        "estrategia_marketing",
        "definir_escada_valor",
        "diagnosticar_gargalo",
        "otimizar_campanha_complexa",
        "analise_competitiva",
        "criar_funil_do_zero"
      ]
    },
    "external_api": {
      "descricao": "APIs especializadas: TTS, imagem, vídeo (não são LLMs de texto)",
      "custo_relativo": "variável",
      "modelos_preferidos": ["elevenlabs", "dall-e-3", "heygen"],
      "tarefas": [
        "gerar_audio",
        "gerar_imagem",
        "gerar_video"
      ]
    }
  },
  "fallback": "inherit",
  "regra_ouro": "NUNCA usar tier pro/standard para tarefa que tier lite resolve"
}

```

Config por Harness (config/harness_profiles.json)

```

{
  "mimocode": {
    "detect": [".mimocode", "mimocode.json"],
    "modelos_conhecidos": {
      "lite": ["mimo-v2.5-lite"],
      "standard": ["mimo-v2.5", "mimo-v2.5-standard"],
      "pro": ["mimo-v2.5-pro"]
    },
    "subagent_model_param": true,
    "actor_models_cmd": "actor models"
  },
  "claude_code": {
    "detect": [".claude", "CLAUDE.md"],
    "modelos_conhecidos": {
      "lite": ["claude-haiku", "claude-3-5-haiku-20241022"],

```



```

        "standard": ["claude-sonnet-4-20250514", "claude-3-5-sonnet-20241022"],
        "pro": ["claude-opus-4-20250514"]
    },
    "subagent_model_param": true,
    "env_key": "ANTHROPIC_API_KEY"
},
"antigravity": {
    "detect": [".antigravity", "antigravity.json"],
    "modelos_conhecidos": {
        "lite": ["antigravity-lite"],
        "standard": ["antigravity-standard"],
        "pro": ["antigravity-pro"]
    },
    "subagent_model_param": true
},
"opencode": {
    "detect": [".opencode", "opencode.json"],
    "modelos_conhecidos": {
        "lite": ["gpt-4o-mini"],
        "standard": ["gpt-4o"],
        "pro": ["o3"]
    },
    "subagent_model_param": true
},
"omp": {
    "detect": [".omp", "omp.json", ".ohmypi"],
    "modelos_conhecidos": {
        "lite": ["mistral-small", "llama-3-8b", "gpt-4o-mini", "claude-haiku"],
        "standard": ["mistral-large", "llama-3-70b", "gpt-4o", "claude-sonnet",
"mimo-v2.5"],
        "pro": ["deepseek-r1", "qwen-max", "o3", "claude-opus", "mimo-v2.5-pro"]
    },
    "subagent_model_param": true,
    "multi_provider": true,
    "nota": "OMP suporta multi-provider via config – detecta por env vars se
config ausente"
},
"gemini_cli": {
    "detect": [".gemini", "gemini.json", ".gemini-cli"],
    "modelos_conhecidos": {
        "lite": ["gemini-2.0-flash", "gemini-2.5-flash"],
        "standard": ["gemini-2.5-pro"],
        "pro": ["gemini-ultra", "gemini-2.5-pro-deep-think"]
    },
    "subagent_model_param": true,
    "env_key": "GOOGLE_API_KEY",
    "image_gen": "imagen-3"
},
"grok": {

```

```

    "detect": [".grok", "grok.json", ".xai"],
    "modelos_conhecidos": {
        "lite": ["grok-2-mini"],
        "standard": ["grok-2", "grok-3"],
        "pro": ["grok-3-heavy"]
    },
    "subagent_model_param": true,
    "env_key": "XAI_API_KEY",
    "image_gen": "aurora"
},
"kiro": {
    "detect": [".kiro", "kiro.json"],
    "modelos_conhecidos": {
        "lite": ["amazon-titan-lite", "claude-haiku"],
        "standard": ["claude-sonnet-v2"],
        "pro": ["claude-opus-v2"]
    },
    "subagent_model_param": true,
    "infra": "aws_bedrock",
    "env_key": "AWS_ACCESS_KEY_ID",
    "nota": "Kiro roda via AWS Bedrock – modelos Anthropic + Amazon Titan"
}
}

```

3. ESTRUTURA (Diretórios)

```

vendas/
├── marketing/
│   ├── maquinas/
│   │   ├── {slug-obra-1}/
│   │   │   ├── manifesto.json # Config, estado, modelos roteados
│   │   │   ├── funis/         # Funis desta máquina
│   │   │   ├── paginas/       # HTML venda/captura
│   │   │   ├── emails/        # Sequências de e-mails
│   │   │   ├── mensagens/     # Sequências DM/WhatsApp
│   │   │   ├── artes/         # Posts, stories, capas
│   │   │   ├── campanhas/     # Cronograma e configs
│   │   │   ├── leads/         # Base de leads (CSV/SQLite)
│   │   │   ├── audio/         # Áudio narrado (podcast/audiobook)
│   │   │   ├── videos/        # Reels, shorts, clips
│   │   │   └── analytics/      # Dashboards e relatórios
│   │   ├── {slug-obra-2}/
│   │   └── ...
│   └── _shared/                # Templates e configs compartilhados
├── leads_global/               # Base mestra de leads (todas as máquinas)
├── scripts/
│   ├── criar-maquina-vendas.py # Orquestrador: cria máquina a partir da obra
│   ├── descobrir_modelos.py    # Detecta harness e lista LLMs disponíveis
│   └── rotear_subagente.py      # Roteia subagente para modelo mais barato

```

- | — gerar-pagina-venda.py
- | — gerar-pagina-captura.py
- | — gerar-arte-divulgacao.py
- | — gerar-sequencia-dm.py
- | — gerar-audio-narrado.py # TTS via API especializada
- | — gerar-video-reels.py # Gera clips curtos para redes
- | — buscar-leads-instagram.py
- | — enviar-emails.py
- | — qualificar-leads.py # Scoring automático de leads
- | — monitorar-funil.py
- | — escalar-campanha.py
- | — sincronizar-crm.py
- | — gerar-relatorio-vendas.py
- | — config/
 - | — escada_valor_default.json
 - | — produtos.json
 - | — personas.json
 - | — canais.json
 - | — cronograma.json
 - | — roteamento_modelos.json # Tier → modelo mais barato
 - | — harness_profiles.json # Perfis de harness conhecidos
 - | — tts_config.json # Configuração de vozes/LLMs para áudio
 - | — subagentes_marketing.json # Registry de subagentes especializados
- | — templates/
 - | — template_pagina_venda.html
 - | — template_pagina_captura.html
 - | — template_email.html
 - | — template_post_social.html
 - | — template_story.html
 - | — template_video_reels.html
 - | — template_audio_intro.html

4. ESCADA DE VALOR (Russel Brunson)

Nível	Tipo	Preço	Produtos
0	Isca Digital (Grátis)	R\$ 0	Lead Magnets, E-books resumidos (capítulo 1 grátis), Checklist/Cheat Sheet, Mini-curso em 3 e-mails
1	Tripwire	R\$ 27-47	E-book completo, Playbook pronto, Deck de apresentação + áudio narrado
2	Produto Core	R\$ 97-197	Livro completo (PDF + EPUB), TCC formatado + orientações, Artigo científico revisado
3	Obra Completa	Até R\$ 297	Coleção completa (livro + ebook + playbook + deck + áudio + artes), Mega-livro compilado, Pacote TCC + Artigo + Orientação + Narrado

Regra: O nível máximo (Obra Completa) inclui TODOS os formatos derivados da obra — PDF, EPUB, áudio narrado, deck, playbook, lead magnet, artes para redes. É o pacote definitivo.

5. FUNIS DE VENDA

Funil A — Funil de Atração (Cold Traffic)

[Anúncio/Post] → [Página de Captura c/ Lead Magnet]
→ [Página de Obrigado c/ Tripwire]
→ [E-mail 1: entrega do lead magnet]
→ [E-mail 2: história + prova social]
→ [E-mail 3: apresentação do Core]
→ [E-mail 4: oferta com urgência]
→ [E-mail 5: último chamado]
→ [Página de Venda do Core]
→ [Upsell Obra Completa]
→ [Downsell E-book]

Funil B — Funil de Nutrição (Warm Traffic)

[Conteúdo no Instagram/YouTube] → [DM automático c/ Lead Magnet]
→ [Sequência de 7 mensagens]
→ [Mensagem 1: entrega + pergunta]
→ [Mensagem 2: compartilhar resultado]
→ [Mensagem 3: apresentar problema]
→ [Mensagem 4: solução = produto core]

- [Mensagem 5: prova social]
- [Mensagem 6: oferta]
- [Mensagem 7: urgência/escassez]

Funil C — Funil de Reativação (Base Existente)

- [E-mail de reengajamento] → [Nova oferta]
 - [Sequência de reativação em 3 e-mails]
 - [Pesquisa de satisfação]
 - [Oferta personalizada baseada na resposta]

6. ARTEFATOS A GERAR

Artefato	Script	Template	Output
Página de Venda	gerar-pagina-venda.py	template_pagina_venda.html	maquinas/{slug}/paginas/venda.html
Página de Captura	gerar-pagina-captura.py	template_pagina_captura.html	maquinas/{slug}/paginas/captura.html
Sequência de E-mails (5-7)	gerar-sequencia-emails.py	template_email.html	maquinas/{slug}/emails/
Sequência de DM (7 msgs)	gerar-sequencia-dm.py	—	maquinas/{slug}/mensagens/
Posts Instagram (15/mês)	gerar-arte-divulgacao.py	template_post_social.html	maquinas/{slug}/artes/posts/
Stories (10/mês)	gerar-arte-divulgacao.py	template_story.html	maquinas/{slug}/artes/stories/
Áudio Narrado (audiobook/podcast)	gerar-audio-narrado.py	—	maquinas/{slug}/audio/
Vídeos Reels/Shorts (5/mês)	gerar-video-reels.py	template_video_reels.html	maquinas/{slug}/videos/
Artes de Capa Produto	gerar-capa.py (existente)	—	maquinas/{slug}/artes/capa/
Cronograma de Campanha	gerar-cronograma.py	—	maquinas/{slug}/campanhas/
Relatório de Vendas	gerar-relatorio-vendas.py	—	maquinas/{slug}/analytics/

7. SUBAGENTES ESPECIALIZADOS COM ROTEAMENTO DE LLM

Cada subagente é roteado automaticamente para o **modelo mais barato disponível** no harness.

7.1 Subagente Narrador de Áudio

Propriedade	Valor
Nome	subagente-narrador-audio
Função	Converte obra completa em áudio narrado (audiobook + podcast)
Tier de LLM	external_api — sempre API especializada (não LLM de texto)
API Preferida	ElevenLabs > OpenAI TTS > Google Cloud TTS
SPEC	SPEC_AUDIO_NARRADO.md
SKILL	narrador-audio
CONFIG	config/tts_config.json
HOOK	hook_pos_criar_audio.json
Output	maquinas/{slug}/audio/ — arquivos .mp3 por capítulo + playlist completa

SPEC_AUDIO_NARRADO.md: - Voz selecionada por tom da obra (técnico, acadêmico, comercial) - Capítulos individuais + arquivo completo concatenado - Intro/outro com música de fundo (CC0) - Marcadores de capítulo para podcast players - Metadados ID3 (título, autor, capa) - Normalização de volume (LUFS -16) - Formato: MP3 192kbps mínimo

7.2 Subagente Criador de Vídeo

Propriedade	Valor
Nome	subagente-criador-video
Função	Gera vídeos curtos (Reels/Shorts/TikTok) a partir do conteúdo
Tier de LLM	external_api — HeyGen/Synthesia (avatar) ou Remotion (programático)
SPEC	SPEC_VIDEO_REELS.md
SKILL	criador-video
CONFIG	config/video_config.json
HOOK	hook_pos_criar_video.json
Output	maquinas/{slug}/videos/ — arquivos .mp4

7.3 Subagente Designer de Artes

Propriedade	Valor
Nome	subagente-designer-artes
Função	Gera prompts de artes visuais + usa API de imagem
Tier de LLM	standard para prompts + external_api para geração
API Imagem	DALL-E 3 > Midjourney > Ideogram
SPEC	SPEC_ARTES_SOCIAIS.md
SKILL	designer-artes
CONFIG	config/artes_config.json
HOOK	hook_pos_criar_artes.json
Output	maquinas/{slug}/artes/ — .png / .webp

7.4 Subagente Copywriter de Conversão

Propriedade	Valor
Nome	subagente-copywriter
Função	Escreve copy de alta conversão para páginas, e-mails, anúncios
Tier de LLM	standard — Claude Sonnet / MiMo V2.5 / GPT-4o
SPEC	SPEC_COPY_CONVERSAO.md
SKILL	copywriter-conversao
CONFIG	config/copy_config.json
HOOK	hook_pos_criar_copy.json
Output	maquinas/{slug}/paginas/, maquinas/{slug}/emails/

7.5 Subagente Qualificador de Leads

Propriedade	Valor
Nome	subagente-qualificador-leads
Função	Busca, qualifica e pontua leads automaticamente
Tier de LLM	lite — Claude Haiku / MiMo Lite / GPT-4o Mini
SPEC	SPEC_QUALIFICACAO_LEADS.md
SKILL	qualificador-leads
CONFIG	config/leads_config.json
HOOK	hook_qualificacao_leads.json
Output	maquinas/{slug}/leads/ — leads_qualificados.csv

7.6 Subagente Analista de Funil

Propriedade	Valor
Nome	subagente-analista-funil
Função	Monitora métricas, identifica gargalos, sugere correções
Tier de LLM	standard — Claude Sonnet / MiMo V2.5
SPEC	SPEC_ANALISE_FUNIL.md
SKILL	analista-funil
CONFIG	config/analytics_config.json
HOOK	hook_analise_funil.json
Output	maquinas/{slug}/analytics/ — relatórios .md + .pdf

7.7 Subagente Campanha de E-mail

Propriedade	Valor
Nome	subagente-campanha-email
Função	Gerencia envio, automação e otimização de e-mails
Tier de LLM	lite — Claude Haiku / MiMo Lite (geração de variações)
SPEC	SPEC_CAMPANHA_EMAIL.md
SKILL	campanha-email
CONFIG	config/email_config.json
HOOK	hook_campanha_email.json
Output	maquinas/{slug}/emails/ — sequências + relatórios

7.8 Subagente Gestor de Tráfego

Propriedade	Valor
Nome	subagente-gestor-trafego
Função	Cria e otimiza anúncios pagos (Meta, Google)
Tier de LLM	standard — estratégia + lite — variações de copy
SPEC	SPEC_GESTOR_TRAFEGO.md
SKILL	gestor-trafego
CONFIG	config/trafego_config.json
HOOK	hook_gestor_trafego.json
Output	maquinas/{slug}/campanhas/ads/

Resumo de Roteamento por Subagente

Subagente	Tier	Modelo Preferido	Modelo Alternativo	Custo Relativo
Narrador de Áudio	external_api	ElevenLabs	OpenAI TTS / Google TTS	Médio
Criador de Vídeo	external_api	HeyGen	Remotion / Aurora (Grok)	Alto
Designer de Artes	standard + external	DALL-E 3	Imagen-3 (Gemini) / Midjourney	Baixo
Copywriter	standard	Claude Sonnet	MiMo V2.5 / Grok-2 / Gemini Pro	Baixo
Qualificador de Leads	lite	Claude Haiku	MiMo Lite / Grok-2 Mini / Gemini Flash	Muito baixo
Analista de Funil	standard	Claude Sonnet	MiMo V2.5 / Grok-3 / Mistral Large	Baixo
Campanha de E-mail	lite	Claude Haiku	MiMo Lite / Titan Lite / Gemini Flash	Muito baixo
Gestor de Tráfego	standard + lite	Claude Sonnet	MiMo V2.5 / Grok-2 / Gemini Pro	Médio

Roteamento por Harness (qual modelo sai de cada um)

Harness	Lite (scoring)	Standard (copy)	Pro (estratégia)	Imagem	Áudio
MiMoCode	mimo-v2.5-lite	mimo-v2.5	mimo-v2.5-pro	DALL-E 3	ElevenLabs
Claude Code	claude-haiku	claude-sonnet	claude-opus	DALL-E 3	ElevenLabs
Antigravity	antigravity-lite	antigravity-standard	antigravity-pro	DALL-E 3	ElevenLabs
OpenCode	gpt-4o-mini	gpt-4o	o3	DALL-E 3	OpenAI TTS
OMP	mistral-small	mistral-large	deepseek-r1	DALL-E 3	ElevenLabs
Gemini CLI	gemini-flash	gemini-2.5-pro	gemini-ultra	Imagen-3	Google TTS
Grok	grok-2-mini	grok-2	grok-3-heavy	Aurora	xAI TTS
Kiro	titan-lite	claude-sonnet-v2	claude-opus-v2	Titan Image	Amazon Polly

Regra de ouro: NUNCA usar tier pro ou standard para tarefa que tier lite resolve.

8. ESTRATÉGIAS

Aquisição de Leads

1. **Instagram (MCP):** Busca ativa por perfis interessados no tema → DM automático oferecendo lead magnet
2. **Conteúdo Orgânico:** Posts diários extraídos do livro/capítulos → CTA para página de captura
3. **SEO:** Páginas de captura otimizadas para busca → tráfego passivo
4. **Parcerias:** Co-marketing com produtores de conteúdo adjacentes
5. **Tráfego Pago:** Meta Ads + Google Ads (gestor de tráfego autônomo)

Nutrição

1. **E-mail Marketing:** Sequência de 5-7 e-mails com storytelling + prova social
2. **DM Marketing:** Sequência de 7 mensagens no Instagram/WhatsApp
3. **Conteúdo Recorrente:** 15 posts/mês + 5 vídeos/mês extraídos dos materiais
4. **Áudio:** Podcast/audiobook como conteúdo de longa duração

Conversão

1. **Urgência Real:** Ofertas com prazo real (não fake)
2. **Prova Social:** Depoimentos, números de download
3. **Risk Reversal:** Garantia de 7-30 dias

4. **Order Bumps:** Adicionar e-book/playbook no checkout
5. **Retargeting:** Pixel + anúncios para visitantes que não converteram

Retenção/Escala

1. **Upsell Sequencial:** Após compra do core → oferecer Obra Completa
2. **Programa de Afiliados:** Comissão para quem indicar
3. **Reativação:** E-mail mensal para base inativa
4. **Cross-sell:** Máquinas diferentes se referenciam (obra A recomenda obra B)

9. SKILLS

Skills de Orquestração

Skill	Função	Disparo
criar-maquina-vendas	Cria máquina completa a partir de obra finalizada	Pergunta pós-obra (automática)
marketing-strategist	Define escada de valor e funil para cada obra	/criar-funil <slug>
campanha-launch	Orquestra lançamento completo de 30 dias	/lançar <slug>

Skills de Geração de Artefatos

Skill	Função	Disparo
pagina-venda	Gera página de venda completa (HTML)	/criar-pagina-venda <slug>
pagina-captura	Gera página de captura com lead magnet	/criar-pagina-captura <slug>
email-sequence	Gera sequência de e-mails de nutrição/venda	/criar-sequencia-emails <slug>
dm-sequence	Gera sequência de mensagens DM	/criar-sequencia-dm <slug>
arte-social	Gera artes para posts/stories	/criar-artes <slug>
narrador-audio	Gera áudio narrado da obra	/criar-audio <slug>
criador-video	Gera vídeos curtos para redes	/criar-videos <slug>
copywriter-conversao	Gera copy de alta conversão	/criar-copy <slug>

Skills de Operação

Skill	Função	Disparo
lead-hunter	Busca e qualifica leads via Instagram	/buscar-leads <tema>
qualificador-leads	Scoring automático de leads	/qualificar-leads <slug>
campanha-email	Gerencia envio de e-mails	/enviar-emails <slug>
gestor-trafego	Cria/otimiza anúncios pagos	/gerir-trafego <slug>

Skills de Monitoramento

Skill	Função	Disparo
monitor-funil	Monitora métricas e sugere ajustes	/monitorar-funil <slug>
analista-funil	Análise profunda com relatórios	/analisar-funil <slug>
auto-correct	Corrige páginas/e-mails com baixa conversão	/corrigir-funil <slug>

10. SPECS

SPECS de Artefatos

SPEC	Descrição
SPEC_PAGINA_VENDA.md	Headline hook-story-offer, dor, solução, prova social, stack de valor, preço âncora, garantia, CTA, FAQ, urgência
SPEC_PAGINA_CAPTURA.md	Headline magnética, bullets, formulário mínimo, prova social, preview lead magnet, sem navegação
SPEC_FUNIL.md	Escada de valor, sequências e-mail/DM, páginas, métricas por etapa, critérios auto-correção
SPEC_CAMPANHA.md	Calendário 30 dias, posts/dia, horários, hash-tags, CTAs, A/B tests

SPECS de Subagentes

SPEC	Descrição	Tier LLM
SPEC_AUDIO_NARRADO.md	Voz por tom, capítulos, intro/outro, ID3, LUFS, MP3 192kbps	external_api
SPEC_VIDEO_REELS.md	15s/30s/60s, 9:16, hook 3s, legendas, avatar, 5 variações	external_api
SPEC_ARTES_SOCIAIS.md	Identidade visual, feed/stories, 15+10/mês, cores da capa	standard + external
SPEC_COPY_CONVERSAO.md	Hook-Story-Offer, 3 headlines, bullets, prova social, CTA	standard
SPEC_QUALIFICACAO_LEADS.md	Scoring 0-100, fontes, classificação, dedup, LGPD	lite
SPEC_ANALISE_FUNIL.md	Métricas, benchmarks, gargalos, A/B, relatório, alertas	standard
SPEC_CAMPANHA_EMAIL.md	Boas-vindas, nutrição, venda, reativação, A/B, rate limit, LGPD	lite
SPEC_GESTOR_TRAFEGO.md	Meta/Google, CBO/ABO, segmentação, 9 criativos, pausa/escala	standard

11. HOOKS (Automação)

Hook 1 – Pergunta Pós-Obra (Trigger: obra_finalizada)

```
{
  "trigger": "obra_finalizada",
  "condicao": "auditar_obra(slug) == 'conforme'",
  "actions": [
    "exibir_pergunta_criar_maquina(slug)",
    "se_sim: criar_maquina_vendas(slug)",
    "se_parcial: exibir_menu_etapas(slug)",
    "se_nao: salvar_obra_fim(slug)"
  ]
}
```

Hook 2 – Criação da Máquina (Trigger: maquina_criada)

```
{
  "trigger": "maquina_criada",
  "pre_requisito": "descobrir_modelos() → rotear_subagentes()",
  "actions_subagentes": [
```

```

    {"subagente": "copywriter", "tier": "standard", "tarefa":
"gerar_pagina_venda(slug)"},
    {"subagente": "copywriter", "tier": "standard", "tarefa":
"gerar_pagina_captura(slug)"},
    {"subagente": "copywriter", "tier": "standard", "tarefa":
"gerar_sequencia_emails(slug)"},
    {"subagente": "copywriter", "tier": "standard", "tarefa":
"gerar_sequencia_dm(slug)"},
    {"subagente": "designer-artes", "tier": "standard+external", "tarefa":
"gerar_artes(slug, 15)"},
    {"subagente": "narrador-audio", "tier": "external_api", "tarefa":
"gerar_audio(slug)"},
    {"subagente": "criador-video", "tier": "external_api", "tarefa":
"gerar_videos(slug, 5)"}
  ],
  "actions_scripts": [
    "atualizar_escada_valor(slug)",
    "criar_cronograma_lancamento(slug)",
    "criar_manifesto_maquina(slug)"
  ]
}

```

Hook 3 — Operação 24/7 (Cron)

```

{
  "triggers_cron": [
    {
      "nome": "lead_hunter",
      "cron": "0 8,14,20 * * *",
      "subagente": "qualificador-leads",
      "tier": "lite",
      "tarefa": "buscar_e_qualificar_leads(todas_maquinas_ativas)"
    },
    {
      "nome": "envio_emails",
      "cron": "0 9 * * *",
      "subagente": "campanha-email",
      "tier": "lite",
      "tarefa": "processar_fila_emails(todas_maquinas_ativas)"
    },
    {
      "nome": "monitoramento",
      "cron": "0 7 * * *",
      "subagente": "analista-funil",
      "tier": "standard",
      "tarefa": "gerar_relatorio_diario(todas_maquinas_ativas)"
    },
    {
      "nome": "publicacao_conteudo",
      "cron": "0 10,18 * * *",

```

```

        "subagente": "designer-artes",
        "tier": "standard",
        "tarefa": "publicar_conteudo_agendado(todas_maquinas_ativas)"
    }
]
}

```

Hook 4 — Auto-Correção (Trigger: métrica abaixo do threshold)

```

{
    "trigger": "conversao_abaixo_threshold",
    "threshold": {"captura": 0.02, "email_open": 0.20, "venda": 0.01},
    "actions": [
        {"subagente": "analista-funil", "tier": "standard", "tarefa":
"diagnosticar_gargalo(maquina_id)"},
        {"subagente": "copywriter", "tier": "standard", "tarefa":
"gerar_variacao_ab(etapa_gargalo)"},
        "aguardar(48h)",
        {"subagente": "analista-funil", "tier": "standard", "tarefa":
"avaliar_variacoes()"},
        "aplicar_vencedora()"
    ]
}

```

Hook 5 — Escala (Trigger: ROAS positivo por 7 dias)

```

{
    "trigger": "roas_positivo_7dias",
    "condicao": "roas_medio >= 2.0 AND dias_consecutivos >= 7",
    "actions": [
        {"subagente": "gestor-trafego", "tier": "standard", "tarefa":
"aumentar_budget(20%)"},
        {"subagente": "gestor-trafego", "tier": "standard", "tarefa":
"criar_lookalike_audience()"},
        {"subagente": "analista-funil", "tier": "standard", "tarefa":
"sugerir_novos_produtos(maquina_id)"}
    ]
}

```

12. MCPS

MCPS Existentes (reaproveitados)

MCP	Uso na Máquina de Vendas
db_state	Armazena estado de cada máquina, leads, métricas
file_writer	Grava páginas HTML, e-mails, artes
pdf_gen	Gera PDFs de relatórios e materiais

MCPS Novos

MCP	Função	Justificativa
mcp_instagram	API do Instagram (Graph API)	Busca de leads, envio de DMs, postagem, métricas
mcp_email	SMTP/SendGrid/Mailchimp	Envio de e-mails em massa, tracking aberturas/cliques
mcp_analytics	Google Analytics + Meta Pixel	Tracking de conversão, atribuição
mcp_payments	Stripe/PagSeguro/Kiwify	Processamento de pagamentos, webhooks
mcp_crm	HubSpot/Notion DB como CRM	Pipeline de leads, histórico de interações
mcp_social_scheduler	Buffer/Later ou API direta	Agendamento de posts, métricas de engajamento
mcp_ai_images	DALL-E 3 / Midjourney API	Geração de artes visuais para posts e capas
mcp_tts	ElevenLabs / OpenAI TTS	Geração de áudio narrado (audiobook/podcast)
mcp_video	HeyGen / Remotion	Geração de vídeos curtos com avatar ou texto animado

13. CONFIG

config/roteamento_modelos.json

```
{
  "_descricao": "Roteamento de LLMs por tier. O sistema detecta o harness e usa o modelo mais barato.",
  "tiers": {
    "lite": {
      "modelos_preferidos": [
        "claude-haiku", "mimo-v2.5-lite", "gpt-4o-mini", "gemini-flash",
        "grok-2-mini", "amazon-titan-lite", "mistral-small"
      ],
      "tarefas": ["qualificar_leads", "scoring", "classificar", "deduplicar", "formatar"]
    },
    "standard": {
      "modelos_preferidos": [
        "claude-sonnet", "mimo-v2.5", "gpt-4o", "gemini-pro",
        "grok-2", "claude-sonnet-v2", "mistral-large"
      ],
    },
  }
}
```

```

    "tarefas": ["gerar_copy", "escrever_emails", "analise_funil",
"gerar_prompt_arte"]
  },
  "pro": {
    "modelos_preferidos": [
      "claude-opus", "mimo-v2.5-pro", "o3",
      "grok-3-heavy", "claude-opus-v2", "deepseek-r1"
    ],
    "tarefas": ["estrategia_marketing", "diagnosticar_gargalo",
"otimizar_campanha"]
  },
  "external_api": {
    "modelos_preferidos": ["elevenlabs", "dall-e-3", "imagen-3", "aurora",
"heygen"],
    "tarefas": ["gerar_audio", "gerar_imagem", "gerar_video"]
  }
},
"fallback": "inherit",
"regra_ouro": "NUNCA usar tier pro/standard para tarefa que tier lite resolve"
}

```

config/harness_profiles.json

```

{
  "mimocode": {
    "detect": [".mimocode"],
    "models": {"lite": ["mimo-v2.5-lite"], "standard": ["mimo-v2.5"], "pro":
["mimo-v2.5-pro"]}
  },
  "claude_code": {
    "detect": [".claude"],
    "models": {"lite": ["claude-haiku"], "standard": ["claude-sonnet"], "pro":
["claude-opus"]}
  },
  "antigravity": {
    "detect": [".antigravity"],
    "models": {"lite": ["antigravity-lite"], "standard": ["antigravity-
standard"], "pro": ["antigravity-pro"]}
  },
  "omp": {
    "detect": [".omp", ".ohmypi"],
    "models": {
      "lite": ["mistral-small", "llama-3-8b", "gpt-4o-mini"],
      "standard": ["mistral-large", "llama-3-70b", "gpt-4o"],
      "pro": ["deepseek-r1", "qwen-max", "o3"]
    }
  },
  "multi_provider": true,
  "nota": "OMP roda qualquer provider – detecta por env vars (OPENAI,
ANTHROPIC, GOOGLE, XAI)"
}

```

```

    "gemini_cli": {
        "detect": [".gemini", ".gemini-cli"],
        "models": {"lite": ["gemini-2.0-flash", "gemini-2.5-flash"], "standard":
["gemini-2.5-pro"], "pro": ["gemini-ultra"]},
        "env_key": "GOOGLE_API_KEY"
    },
    "grok": {
        "detect": [".grok", ".xai"],
        "models": {"lite": ["grok-2-mini"], "standard": ["grok-2", "grok-3"], "pro":
["grok-3-heavy"]},
        "env_key": "XAI_API_KEY"
    },
    "kiro": {
        "detect": [".kiro"],
        "models": {"lite": ["amazon-titan-lite", "claude-haiku"], "standard":
["claude-sonnet-v2"], "pro": ["claude-opus-v2"]},
        "infra": "aws_bedrock",
        "env_key": "AWS_ACCESS_KEY_ID"
    }
}

```

config/subagentes_marketing.json

```

{
    "registry": {
        "narrador-audio": {"tier": "external_api", "auto_triggers":
["obra_finalizada"]},
        "criador-video": {"tier": "external_api", "auto_triggers":
["maquina_criada"]},
        "designer-artes": {"tier": "standard", "auto_triggers": ["maquina_criada",
"cron_diario"]},
        "copywriter": {"tier": "standard", "auto_triggers": ["maquina_criada",
"auto_correcao"]},
        "qualificador-leads": {"tier": "lite", "auto_triggers": ["cron_3x_dia"]},
        "analista-funil": {"tier": "standard", "auto_triggers": ["cron_diario"]},
        "campanha-email": {"tier": "lite", "auto_triggers": ["cron_diario"]},
        "gestor-trafeego": {"tier": "standard", "auto_triggers": ["cron_diario"]}
    }
}

```

config/tts_config.json

```

{
    "provedor": "elevenlabs",
    "voz_padrao": "Rachel",
    "vozes_por_tom": {
        "tecnico": "Adam",
        "academico": "Nicole",
        "comercial": "Rachel",
        "narrativo": "Antoni"
    }
}

```

```

    "velocidade": 1.0,
    "estabilidade": 0.65,
    "formato_saida": "mp3",
    "bitrate": 192,
    "normalizar_lufs": -16
}

config/produtos.json
{
  "catalogo": {
    "gerado_por": "fabrica",
    "mapeamento_funil": {
      "lead_magnet": {"nivel": 0, "preco": 0},
      "ebook": {"nivel": 1, "preco": 37},
      "playbook": {"nivel": 1, "preco": 27},
      "livro": {"nivel": 2, "preco": 97},
      "tcc": {"nivel": 2, "preco": 147},
      "obra_completa": {
        "nivel": 3,
        "preco": 297,
        "inclui": ["livro_pdf", "epub", "audio_narrado", "deck", "playbook",
"lead_magnet", "artes_15"]
      }
    }
  }
}

```

14. AGENTS.md / CLAUDE.md — Seções Novas

Seção 8: Máquina de Vendas Autônoma

Conceito: Após cada obra finalizada, o sistema pergunta se o operador deseja criar uma máquina de vendas autônoma. Cada máquina é independente — com seus próprios funis, leads, campanhas e subagentes.

Roteamento de LLMs: O sistema detecta o harness atual (MiMoCode, Claude Code, Antigravity, etc.) e roteia cada subagente para o modelo mais barato disponível. Tarefas simples usam tier lite, copy usa standard, estratégia usa pro, áudio/vídeo/imagem usam external_api.

Squad de Marketing (por máquina):

marketing-strategist → copywriter (standard) → designer-artes (standard+external) → narrador-audio (external) → criador-video (external) → qualificador-leads (lite) → campanha-email (lite) → gestor-trafego (standard) → analista-funil (standard) → auto-correct

Fluxo:

1. **Trigger:** Obra finalizada + validada → pergunta ao operador
2. **Deteção:** descobrir_modelos() → identifica harness e LLMs disponíveis

3. **Roteamento:** Cada subagente recebe o modelo mais barato do seu tier
4. **Criação:** Subagentes geram em paralelo: páginas, e-mails, DMs, artes, áudio, vídeos
5. **Operação 24/7:** Cron jobs buscam leads, enviam e-mails, publicam conteúdo, monitoram
6. **Auto-correção:** Quando conversão cai, subagentes diagnosticam e corrigem
7. **Escala:** Quando ROAS positivo, aumenta budget e replica funil

Comandos:

- `/criar-maquina <slug>` — cria máquina de vendas para obra existente
 - `/status-maquinas` — dashboard de todas as máquinas ativas
 - `/pausar-maquina <slug>` — pausa operação de uma máquina
 - `/monitorar-funil <slug>` — métricas de uma máquina específica
 - `/corrigir-funil <slug>` — auto-correção baseada em dados
-

15. PLANO DE IMPLEMENTAÇÃO

Prioridade 1 — MVP (Funcional)

1. `criar-maquina-vendas.py` — orquestrador principal
2. `descobrir_modelos.py` — detecção de harness + roteamento
3. Specs: `SPEC_PAGINA_VENDA.md`, `SPEC_PAGINA_CAPTURA.md`, `SPEC_FUNIL.md`
4. Subagentes: copywriter (standard), designer-artes (standard)
5. Scripts: `gerar-pagina-venda.py`, `gerar-pagina-captura.py`, `gerar-sequencia-emails.py`
6. Pergunta pós-obra no fluxo da fábrica

Prioridade 2 — Operação

7. Subagentes: `qualificador-leads (lite)`, `campanha-email (lite)`, `analista-funil (standard)`
8. MCPs: `mcp_instagram`, `mcp_email`
9. Hooks de automação (cron jobs)
10. `config/roteamento_modelos.json`, `config/harness_profiles.json`

Prioridade 3 — Conteúdo Rico

11. Subagentes: `narrador-audio (external)`, `criador-video (external)`
12. MCPs: `mcp_tts`, `mcp_video`
13. `SPEC_AUDIO_NARRADO.md`, `SPEC_VIDEO_REELS.md`
14. Templates de áudio/vídeo

Prioridade 4 — Escala

15. Subagente: `gestor-trafego (standard)`
 16. MCPs: `mcp_analytics`, `mcp_payments`, `mcp_crm`
 17. Auto-correção e A/B testing
 18. Escala automática de budget
 19. Cross-sell entre máquinas
-

16. MANUAL PASSO A PASSO — Como Colocar a Máquina para Funcionar

Pré-requisitos

Antes de começar, verifique:

- ✓ Fábrica de Livros funcionando (consegue criar obras)
- ✓ Python 3.10+ instalado
- ✓ Harness configurado (MiMoCode, Claude Code, ou outro)
- ✓ Pelo menos 1 LLM configurada no harness
- ✓ Conta de e-mail para envios (Gmail/App Password ou SendGrid)
- ✓ Conta no Instagram (para lead hunting via DM)

Passo 1: Detectar seu Harness e LLMs

Rodar o script de detecção

```
python scripts/descobrir_modelos.py
```

Saída esperada:

```
# {
#   "harness": "mimocode",
#   "modelos_disponiveis": {"mimo-v2.5-pro": true, "mimo-v2.5-lite": true},
#   "roteamento": {
#     "qualificar_leads": {"provider": "mimo", "model": "mimo-v2.5-lite"},
#     "gerar_copy": {"provider": "mimo", "model": "mimo-v2.5"},
#     ...
#   }
# }
```

O que acontece: O script detecta qual harness você está usando (MiMoCode, Claude Code, Antigravity, etc.), lista as LLMs disponíveis e cria automaticamente o roteamento — cada tarefa recebe o modelo mais barato capaz.

Passo 2: Configurar APIs Externas (Opcional)

Se quiser áudio, vídeo e artes, configure as APIs:

Copiar o exemplo de .env

```
cp .env.example .env
```

Editar .env com suas chaves:

ELEVENLABS_API_KEY=sua_chave_aqui	# Para áudio narrado
DALL_E_API_KEY=sua_chave_aqui	# Para geração de imagens
HEYGEN_API_KEY=sua_chave_aqui	# Para vídeos com avatar
INSTAGRAM_ACCESS_TOKEN=seu_token_aqui	# Para busca de leads
SENDGRID_API_KEY=sua_chave_aqui	# Para envio de e-mails

Sem essas chaves: A máquina funciona parcialmente — gera textos, páginas HTML e e-mails, mas não gera áudio/vídeo/imagens automaticamente.

Passo 3: Criar uma Obra (ou usar uma existente)

Se já tem uma obra criada:

```
ls output/livros/
```

```
# Se precisa criar uma nova:
/criar-livro "Inteligência Artificial para Empreendedores"
```

Passo 4: Responder “Sim” à Pergunta

Após a obra ser finalizada e validada, o sistema exibe:

Obra "Inteligência Artificial para Empreendedores" concluída.
Deseja criar uma MÁQUINA DE VENDAS autônoma?

- [1] Sim – criar máquina completa
- [2] Sim – criar máquina parcial
- [3] Não – apenas salvar

→ Digite 1

Passo 5: Acompanhar a Criação

O sistema executa automaticamente (em paralelo):

[1/8] Copywriter (standard)	→ Gerando página de venda...	✓
[2/8] Copywriter (standard)	→ Gerando página de captura...	✓
[3/8] Copywriter (standard)	→ Gerando sequência de e-mails...	✓
[4/8] Copywriter (standard)	→ Gerando sequência de DMs...	✓
[5/8] Designer (standard+ext)	→ Gerando 15 artes...	✓
[6/8] Narrador (external_api)	→ Gerando áudio narrado...	⚠ API key não configurada
[7/8] Criador Vídeo (external)	→ Gerando 5 vídeos...	⚠ API key não configurada
[8/8] Orquestrador	→ Criando manifesto...	✓

Máquina criada em: marketing/maquinas/inteligencia-artificial-empreendedores/

Passo 6: Revisar os Artefatos Gerados

```
# Ver o que foi gerado
```

```
ls marketing/maquinas/inteligencia-artificial-empreendedores/
```

```
# Abrir a página de venda no navegador
```

```
start marketing/maquinas/inteligencia-artificial-empreendedores/paginas/
venda.html
```

```
# Revisar e-mails
```

```
cat marketing/maquinas/inteligencia-artificial-empreendedores/emails/
sequencia_01.md
```

```
# Ver manifesto da máquina
```

```
cat marketing/maquinas/inteligencia-artificial-empreendedores/manifesto.json
```

Passo 7: Configurar Canais de Distribuição

7a. E-mail Marketing

```
# Configurar provedor de e-mail em config/email_config.json
{
  "provedor": "sendgrid",
  "api_key_env": "SENDGRID_API_KEY",
  "remetente": "contato@seudominio.com",
  "nome_remetente": "Seu Nome"
}
```

7b. Instagram (Lead Hunting)

```
# Configurar em config/leads_config.json
{
  "fontes": ["instagram"],
  "instagram": {
    "access_token_env": "INSTAGRAM_ACCESS_TOKEN",
    "nicho_hashtags": ["#empreendedorismo", "#ia", "#tecnologia"],
    "dm_template": "marketing/maquinas/{slug}/mensagens/dm_01.md"
  }
}
```

7c. Páginas de Venda/Captura (Hospedagem)

```
# Opção 1: Vercel (gratuito para início)
vercel deploy marketing/maquinas/{slug}/paginas/
```

```
# Opção 2: GitHub Pages
git add marketing/maquinas/{slug}/paginas/
git commit -m "deploy: páginas de venda"
git push origin main
```

```
# Opção 3: Seu próprio servidor
scp -r marketing/maquinas/{slug}/paginas/ user@server:/var/www/html/
```

Passo 8: Ativar a Operação 24/7

```
# Ativar os cron jobs de operação
/ativar-operacao {slug}
```

```
# O que será ativado:
# 🕒 08:00, 14:00, 20:00 – Busca de leads (tier: lite)
# 🕒 09:00 – Envio de e-mails da fila (tier: lite)
# 🕒 07:00 – Relatório diário de métricas (tier: standard)
# 🕒 10:00, 18:00 – Publicação de conteúdo (tier: standard)
```

Passo 9: Monitorar e Ajustar

```
# Ver dashboard de métricas
/monitorar-funil {slug}
```

```
# Relatório diário (gerado automaticamente às 07:00)
```



```
cat marketing/maquinas/{slug}/analytics/relatorio_2026-08-08.md
```

```
# Se a conversão estiver baixa, o sistema auto-corrige:
# 1. Analista diagnostica o gargalo
# 2. Copywriter gera variação A/B
# 3. Após 48h, avalia qual venceu
# 4. Aplica a vencedora automaticamente
```

Passo 10: Escalar quando Houver Retorno

```
# Quando ROAS > 2x por 7 dias consecutivos, o sistema:
# 1. Aumenta budget de anúncios em 20%
# 2. Cria lookalike audience
# 3. Sugere novos produtos para a escada de valor
```

```
# Para escalar manualmente:
/escalar-campanha {slug} --budget +30%
```

Comandos Disponíveis (Resumo)

Comando	O que faz
/criar-maquina <slug>	Cria máquina de vendas para obra existente
/status-maquinas	Dashboard de todas as máquinas ativas
/pausar-maquina <slug>	Pausa operação
/reativar-maquina <slug>	Reativa operação pausada
/monitorar-funil <slug>	Métricas em tempo real
/corrigir-funil <slug>	Força auto-correção
/escalar-campanha <slug>	Aumenta budget/alcance
/relatorio <slug>	Gera relatório sob demanda
/descobrir-modelos	Re-detecta harness e LLMs

Troubleshooting

Problema	Solução
“Nenhum modelo detectado”	Verifique se o harness está configurado (python scripts/descobrir_modelos.py)
“API key não configurada”	Adicione a chave no .env (Passo 2)
“Página não abre”	Verifique se o HTML foi gerado (ls paginas/)
“E-mails não enviam”	Verifique configuração do provedor em config/email_config.json
“Leads não aparecem”	Verifique token do Instagram em config/leads_config.json
“Áudio não gera”	Requer API externa (ElevenLabs/OpenAI TTS) — configure no .env
“Conversão baixa”	O sistema auto-corrigi após 48h, ou force com /corrigir-funil
“Gasto de tokens alto”	Verifique se o roteamento está usando tiers corretos (/descobrir-modelos)
“OMP não detecta providers”	Verifique env vars: OPENAI_API_KEY, ANTHROPIC_API_KEY, GOOGLE_API_KEY
“Gemini CLI sem acesso”	Configure GOOGLE_API_KEY ou GOOGLE_GENAI_API_KEY no .env
“Grok não responde”	Configure XAI_API_KEY no .env (obtido em console.x.ai)
“Kiro/Bedrock com erro”	Verifique AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY e região Bedrock habilitada

Documento atualizado em 2026-08-08 — Fábrica Agêntica de Publicações